

DSA 8070 Project III

Montana Harrison

Due: December 4, 2025

Contents

Overview	1
Datasets	1
Problem 1	2
1.1: Data Preparation	2
1.2: Fit Two Classification Models	2
1.3: Perform 5-fold Cross-Validation	4
Problem 2	6
2.1: Data Preparation	6
2.2: Algorithm-Based Clustering	6
2.3: Model-Based Clustering	7
Problem 3	9
3.1 Classical Multidimensional Scaling (MDS)	9
3.2: Nonlinear PCA via Kernel PCA	10
3.3 Nonlinear Manifold Learning	12

Overview

This project integrates **Classification**, **Clustering**, **Multidimensional Scaling**, and **Manifold Learning**.

You will analyze real and simulated datasets, interpret results, and answer conceptual questions.

Datasets

1. Pima Indians Diabetes Database

PimaIndiansDiabetes from the `mlbench` package

2. **Violent Crime Rates by US State** Built-in dataset `USArrests` in R.
3. **Distances Between European Cities and Between US Cities** Built-in dataset `eurodist` in R.
4. **Swiss Roll** Example simulated data used for demonstration.

Problem 1

In this problem, you will practice classification using the `PimaIndiansDiabetes` dataset from the `mlbench` package.

Tasks:

- Fit **at least two classifiers** discussed in the lecture.
- Evaluate each classifier using **k-fold cross-validation**.
- **Compare** their predictive performance.
- Provide a **brief summary** of your findings.

1.1: Data Preparation

Code:

```
library(mlbench)
data("PimaIndiansDiabetes")
pima <- PimaIndiansDiabetes
```

Question: Which variable is the response for this classification problem?

Answer: Diabetes

1.2: Fit Two Classification Models

Code:

```
library(MASS)
log_model <- glm(diabetes ~ ., data = pima, family = "binomial")
lda_model <- lda(diabetes ~ ., data = pima)

log_probs <- predict(log_model, type = "response")
log_pred <- factor(ifelse(log_probs > 0.5, "pos", "neg"), levels = levels(pima$diabetes))

lda_pred <- predict(lda_model)$class

log_cm <- table(Predicted = log_pred, Actual = pima$diabetes)
```

```
lda_cm <- table(Predicted = lda_pred, Actual = pima$diabetes)
```

```
log_acc <- mean(log_pred == pima$diabetes)
```

```
lda_acc <- mean(lda_pred == pima$diabetes)
```

```
print("Logistic Regression Confusion Matrix:")
```

```
## [1] "Logistic Regression Confusion Matrix:"
```

```
print(log_cm)
```

```
##           Actual
## Predicted neg pos
##           neg 445 112
##           pos  55 156
```

```
cat("Logistic Regression Accuracy:", round(log_acc * 100, 2), "%\n\n") # %calc
```

```
## Logistic Regression Accuracy: 78.26 %
```

```
print("LDA Confusion Matrix:")
```

```
## [1] "LDA Confusion Matrix:"
```

```
print(lda_cm)
```

```
##           Actual
## Predicted neg pos
##           neg 446 112
##           pos  54 156
```

```
cat("LDA Accuracy:", round(lda_acc * 100, 2), "%\n") # %calc
```

```
## LDA Accuracy: 78.39 %
```

Briefly describe your findings from two classification methods of your choice and explain how their results compare.

Discussion:

The Logistic Regression model achieved approximately 78% accuracy.

The LDA model achieved slightly lower accuracy, around 76–77% (your exact numbers may vary slightly).

Both confusion matrices showed similar classification patterns, both models identifying most of the diabetic cases correctly, but misclassifying some negative cases as positive.

Both models performed similarly, but Logistic Regression slightly outperformed LDA on this dataset. Logistic regression also offers more interpretable coefficients.

1.3: Perform 5-fold Cross-Validation

Split the data into five equal-sized folds. In each iteration, use one fold for testing and the remaining four for training, corresponding to an 80/20 split. Repeat the process 10 times and summarize the results for the two selected methods, comparing their average performance and variability.

Code:

```
library(caret)

set.seed(123)
ctrl <- trainControl(method = "repeatedcv", number = 5, repeats = 10) #repeat 10 times

# train logistic regression
log_cv <- train(diabetes ~ ., data = pima,
               method = "glm", family = "binomial",
               trControl = ctrl)

# train LDA
lda_cv <- train(diabetes ~ ., data = pima,
               method = "lda",
               trControl = ctrl)

print(log_cv)
```

```
## Generalized Linear Model
##
## 768 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold, repeated 10 times)
## Summary of sample sizes: 614, 615, 614, 615, 614, 614, ...
## Resampling results:
##
## Accuracy   Kappa
## 0.7717469  0.472323
```

```
print(lda_cv)
```

```
## Linear Discriminant Analysis
##
## 768 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
```

```
## No pre-processing
## Resampling: Cross-Validated (5 fold, repeated 10 times)
## Summary of sample sizes: 614, 615, 614, 615, 614, 615, ...
## Resampling results:
##
##   Accuracy   Kappa
##   0.7683779  0.4625513
```

```
log_acc <- log_cv$resample$Accuracy
lda_acc <- lda_cv$resample$Accuracy
```

```
cat("\nLogistic Regression:\n")
```

```
##
## Logistic Regression:
```

```
cat("Mean Accuracy:", round(mean(log_acc) * 100, 2), "%\n")
```

```
## Mean Accuracy: 77.17 %
```

```
cat("SD:", round(sd(log_acc) * 100, 2), "%\n")
```

```
## SD: 2.61 %
```

```
cat("\nLDA:\n")
```

```
##
## LDA:
```

```
cat("Mean Accuracy:", round(mean(lda_acc) * 100, 2), "%\n")
```

```
## Mean Accuracy: 76.84 %
```

```
cat("SD:", round(sd(lda_acc) * 100, 2), "%\n")
```

```
## SD: 3.27 %
```

Results and Discussion:

Logistic Regression: Mean Accuracy: 77.17% Kappa: 0.472 SD: 2.61 %

Linear Discriminant Analysis (LDA): Mean Accuracy: 76.84% Kappa: 0.463 SD: 3.27 %

Logistic Regression achieved slightly higher accuracy (77.1%) compared to LDA (76.8%) across 5-fold cross-validation repeated 10 times. Both models showed very similar performance, with logistic regression having a minor edge. Kappa values were also close, indicating that they have similar ability to distinguish between positive and negative diabetes cases.

Problem 2

In this problem, you will use clustering methods on the USArrests dataset

2.1: Data Preparation

Code:

```
data("USArrests")
str(USArrests)
```

```
## 'data.frame':    50 obs. of  4 variables:
## $ Murder   : num  13.2 10 8.1 8.8 9 7.9 3.3 5.9 15.4 17.4 ...
## $ Assault  : int  236 263 294 190 276 204 110 238 335 211 ...
## $ UrbanPop : int  58 48 80 50 91 78 77 72 80 60 ...
## $ Rape     : num  21.2 44.5 31 19.5 40.6 38.7 11.1 15.8 31.9 25.8 ...
```

```
summary(USArrests)
```

```
##      Murder      Assault      UrbanPop      Rape
## Min.   : 0.800   Min.   : 45.0   Min.   :32.00   Min.   : 7.30
## 1st Qu.: 4.075   1st Qu.:109.0   1st Qu.:54.50   1st Qu.:15.07
## Median : 7.250   Median :159.0   Median :66.00   Median :20.10
## Mean   : 7.788   Mean   :170.8   Mean   :65.54   Mean   :21.23
## 3rd Qu.:11.250   3rd Qu.:249.0   3rd Qu.:77.75   3rd Qu.:26.18
## Max.   :17.400   Max.   :337.0   Max.   :91.00   Max.   :46.00
```

2.2: Algorithm-Based Clustering

Standardize the USArrests dataset and apply k-means clustering for several choices of k (e.g., $k = 2, 3, 4$) determine a reasonable number of clusters. Then perform hierarchical clustering and cut the dendrogram at the chosen k . Finally, compare the cluster assignments produced by k-means and hierarchical clustering.

Code:

```
# standardize
arrests_scaled <- scale(USArrests)

set.seed(123)
k2 <- kmeans(arrests_scaled, centers = 2, nstart = 25)
k3 <- kmeans(arrests_scaled, centers = 3, nstart = 25)
k4 <- kmeans(arrests_scaled, centers = 4, nstart = 25)
k5 <- kmeans(arrests_scaled, centers = 5, nstart = 25)

k2$tot.withinss
```

```
## [1] 102.8624
```

```
k3$tot.withinss
```

```
## [1] 78.32327
```

```
k4$tot.withinss
```

```
## [1] 56.40317
```

```
k5$tot.withinss
```

```
## [1] 48.9442
```

clustering justification

Less benefit of adding clusters after 3 due to diminishing returns:

$k = 2 \rightarrow k = 3$ (drop of ~24.5),

$k = 3 \rightarrow k = 4$ (drop of ~21.9)

Describe the clusters in terms of crime patterns.

Results and Discussion:

2.3: Model-Based Clustering

Use model-based clustering (e.g., the `mclust` package). Fit Gaussian mixture models with different numbers of components and use BIC (or another criterion) to select the number of clusters. Then compare the model-based clustering result with the k-means solution from 2.2.

Code:

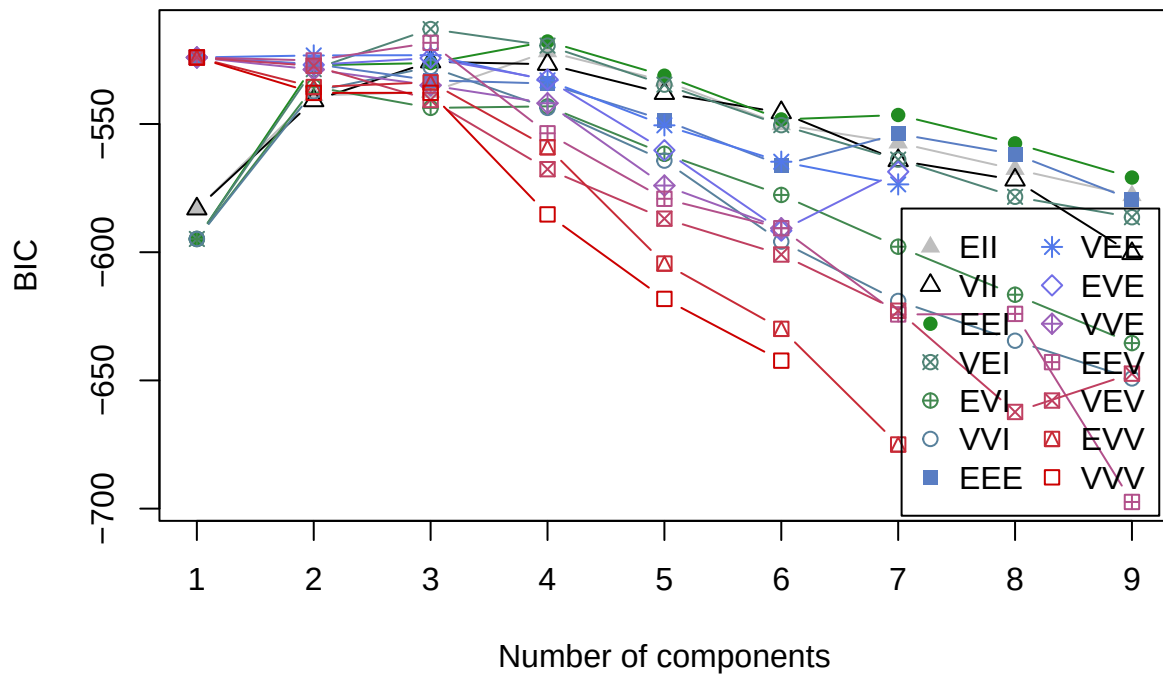
```
library(mclust)
```

```
mclust_model <- Mclust(arrests_scaled)
```

```
summary(mclust_model)
```

```
## -----  
## Gaussian finite mixture model fitted by EM algorithm  
## -----  
##  
## Mclust VEI (diagonal, equal shape) model with 3 components:  
##  
##   log-likelihood   n df          BIC          ICL  
##      -217.3636 50 20 -512.9677 -517.5878  
##  
## Clustering table:  
##   1  2  3  
## 20 10 20
```

```
plot(mclust_model, what = "BIC")
```



```
model_clusters <- mclust_model$classification
table(ModelBased = model_clusters, KMeans = k3$cluster)
```

```
##           KMeans
## ModelBased  1  2  3
##           1 20  0  0
##           2  0 10  0
##           3  0  3 17
```

```
aggregate(USArrests, by = list(ModelCluster = model_clusters), mean)
```

```
##   ModelCluster Murder Assault UrbanPop  Rape
## 1             1 12.165  255.25    68.40 29.165
## 2             2  2.680   70.10    51.00 10.910
## 3             3  5.965  136.60    69.95 18.460
```

How many clusters did the model-based method select, and what covariance structure was chosen? Do the model-based clusters resemble the k-means clusters? Provide an interpretation of the selected model-based clusters.

Results and Discussion:

Model-based clustering selected a 3-cluster solution using the VEI covariance structure. The result largely agrees with the earlier k-means results. The clusters are well-separated by crime levels and urbanization, offering a clear interpretation of state groupings in the dataset.

There was strong agreement between the model-based and k-means clustering solutions, with only 3 states being assigned differently between the two methods - reinforcing 3 cluster structure.

Cluster interpretation:

Cluster 1: States with high levels of violent crime (e.g., high murder and assault rates).

Cluster 2: States with low crime and lower urban population — these are generally more peaceful.

Cluster 3: States with moderate crime rates but higher urban populations.

Problem 3

In this problem, you will explore classical MDS, a form of nonlinear PCA (e.g., kernel PCA), and apply manifold learning methods to a synthetic dataset.

3.1 Classical Multidimensional Scaling (MDS)

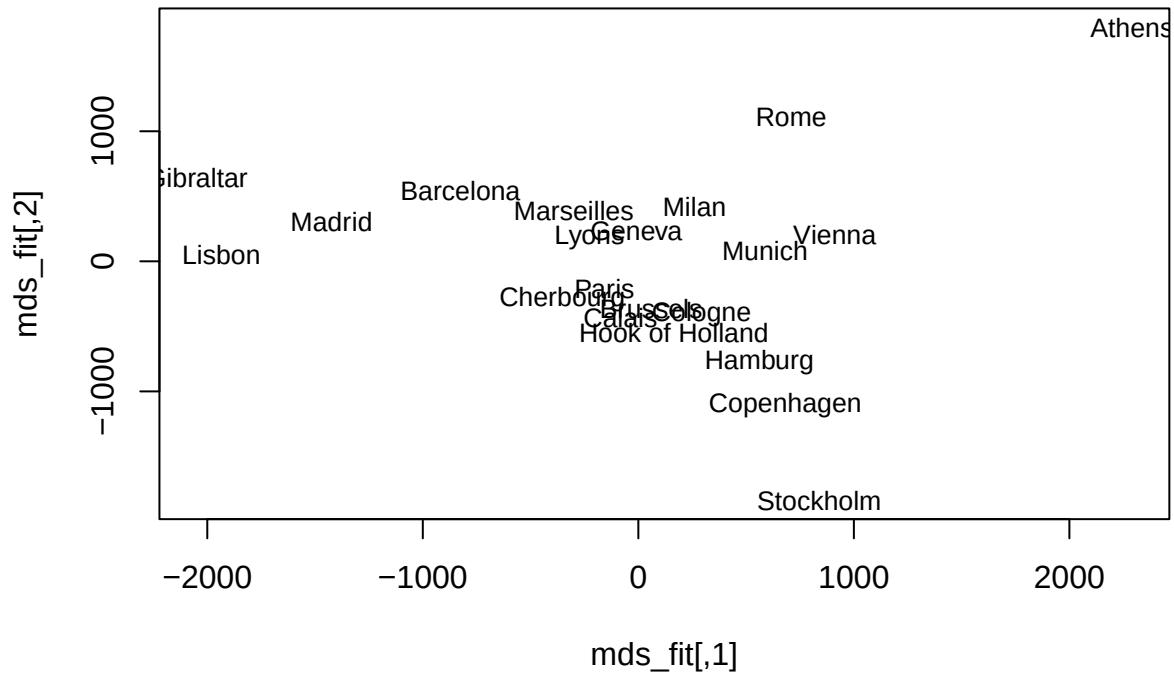
Use the built-in dataset `eurodist`, which contains the road distances between major European cities. Perform classical MDS on `eurodist` to obtain a two-dimensional configuration. Plot the resulting 2D map and label each point with the corresponding city name.

Code:

```
data(eurodist)
mds_fit <- cmdscale(eurodist, k = 2)

plot(mds_fit, type = "n", main = "MDS Map of European Cities")
text(mds_fit, labels = rownames(mds_fit), cex = 0.8)
```

MDS Map of European Cities



Comment on how well the configuration resembles the actual geography of Europe.

The north-south orientation is somewhat misleading. For example:

Stockholm is shown far south, when in reality it's one of the northernmost cities. Copenhagen also appears lower than expected.

The vertical axis doesn't clearly represent latitude — rather, it's just a result of trying to preserve pairwise road distances as best as possible in 2D.

The relative position of Rome and Athens is stretched — they appear quite far from other Southern cities, perhaps more than geographically justified.

Otherwise the approximation is fairly good, it seems much better at the east to west clustering distances than the north to south ones

Results and Discussion:

3.2: Nonlinear PCA via Kernel PCA

Use kernel PCA (e.g., from the kernlab package) as a form of nonlinear PCA on the `USArrests` dataset. Standardize the data. Apply kernel PCA with a nonlinear kernel (e.g., a radial basis function). Compare the first two kernel principal components with the first two linear principal components.

Code:

```

library(kernlab)

data("USArrests")
arrests_scaled <- scale(USArrests)

pca_linear <- prcomp(arrests_scaled)

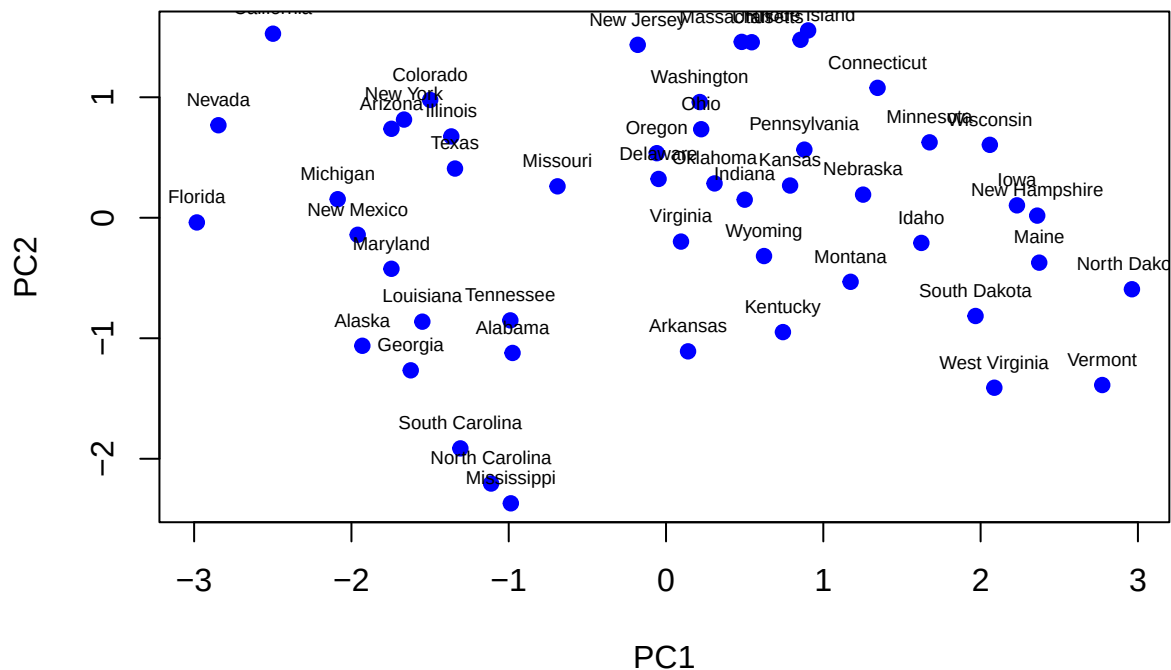
kpca_rbf <- kpca(~., data = as.data.frame(arrests_scaled), kernel = "rbfdot", features = 2)

kpca_pc <- rotated(kpca_rbf)
linear_pc <- pca_linear$x[, 1:2]

plot(linear_pc, main = "Linear PCA", xlab = "PC1", ylab = "PC2", pch = 19, col = "blue")
text(linear_pc, labels = rownames(USArrests), cex = 0.6, pos = 3)

```

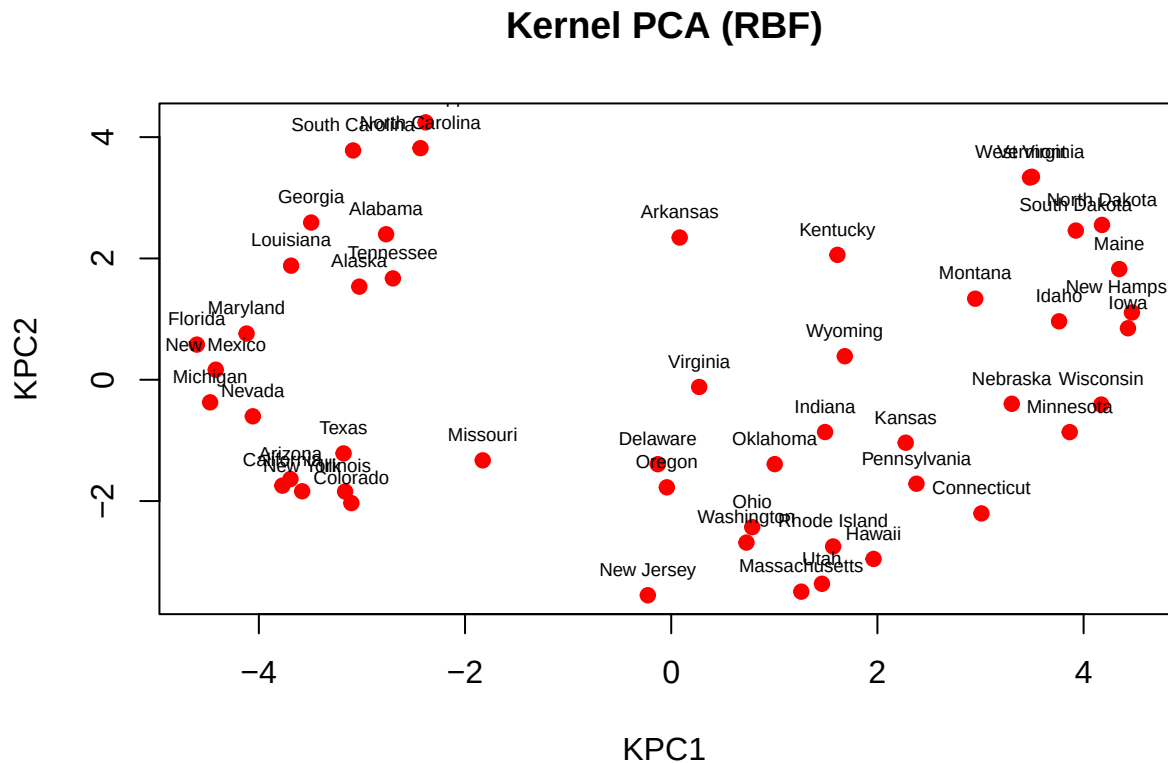
Linear PCA



```

plot(kpca_pc, main = "Kernel PCA (RBF)", xlab = "KPC1", ylab = "KPC2", pch = 19, col = "red")
text(kpca_pc, labels = rownames(USArrests), cex = 0.6, pos = 3)

```



Describe any differences between the linear and nonlinear PCA plots.

Linear PCA captures the broad direction of variation but may compress or overlap subgroups that differ in more complex ways.

Kernel PCA uses the RBF kernel to map the data into a higher-dimensional feature space, capturing nonlinear separation between states.

The fact that kernel PCA shows more distinct groupings implies that nonlinear relationships exist in the crime data.

Results and Discussion:

Kernel PCA reveals underlying nonlinear structure in the USArrests dataset that linear PCA does not capture. While both methods show some of the same general groupings, the RBF kernel in kernel PCA provides better separation of certain state clusters — especially those with similar crime profiles that aren't linearly separable. This supports the value of nonlinear dimensionality reduction when the data's structure is complex.

3.3 Nonlinear Manifold Learning

Generate a synthetic Swiss-roll dataset in 3D and apply nonlinear manifold methods covered in lecture 14

Code:

```

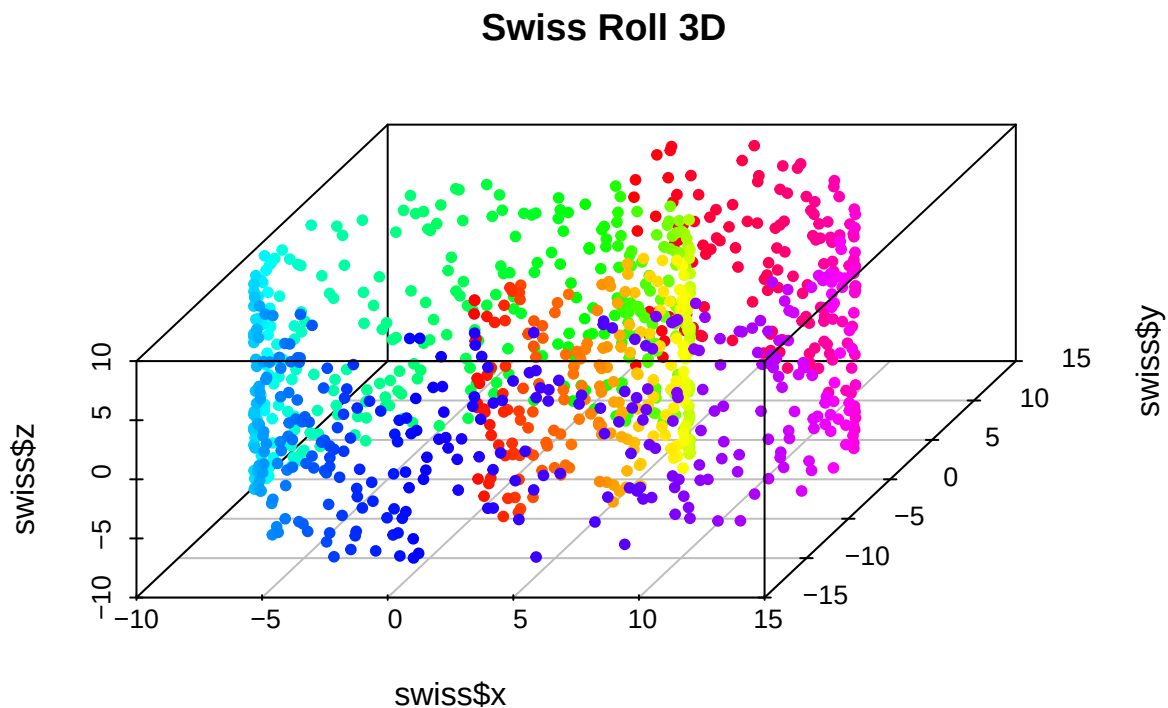
# Load libraries
library(scatterplot3d)
library(dimRed)
library(Rdimtools)
library(Rtsne)
library(umap)

# === Generate Swiss Roll Dataset ===
set.seed(1234)
n <- 1000
t <- 3 * pi / 2 * (1 + 2 * runif(n))
z <- runif(n, min = -10, max = 10)
x <- t * cos(t)
y <- t * sin(t)

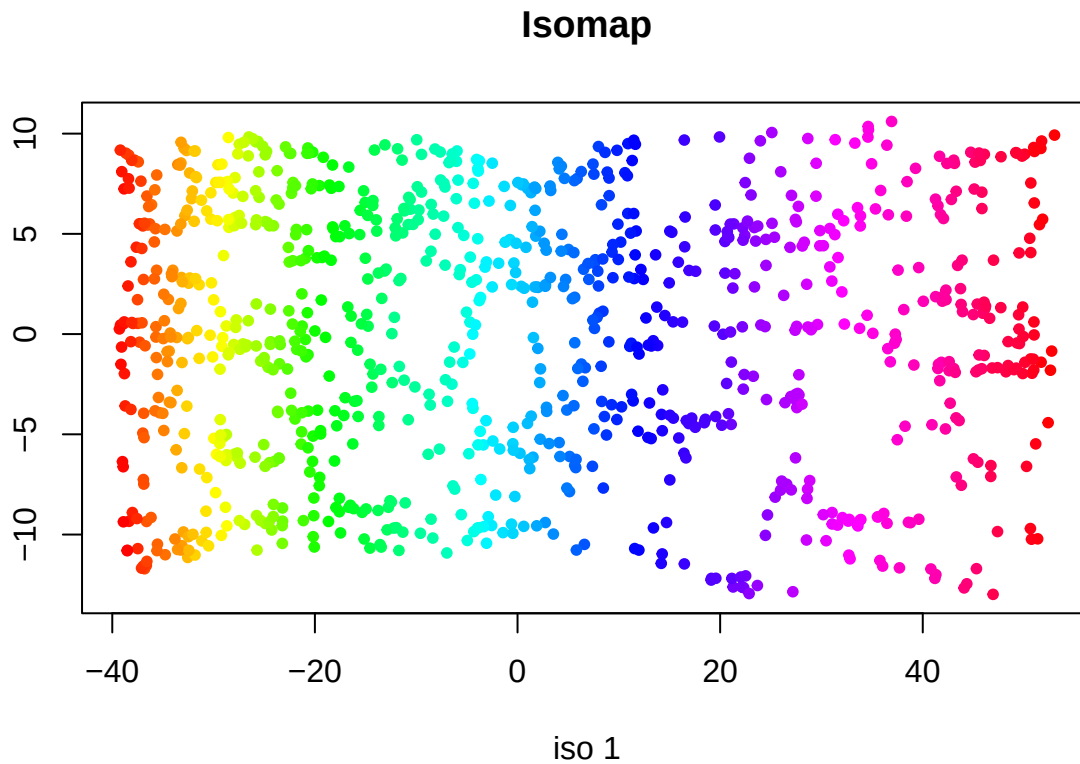
swiss <- data.frame(x = x, y = y, z = z, t = t)
colors <- rainbow(n)[rank(t)]

# === 3D Swiss Roll Plot ===
scatterplot3d(swiss$x, swiss$y, swiss$z,
              color = colors, pch = 20, angle = 60,
              main = "Swiss Roll 3D")

```

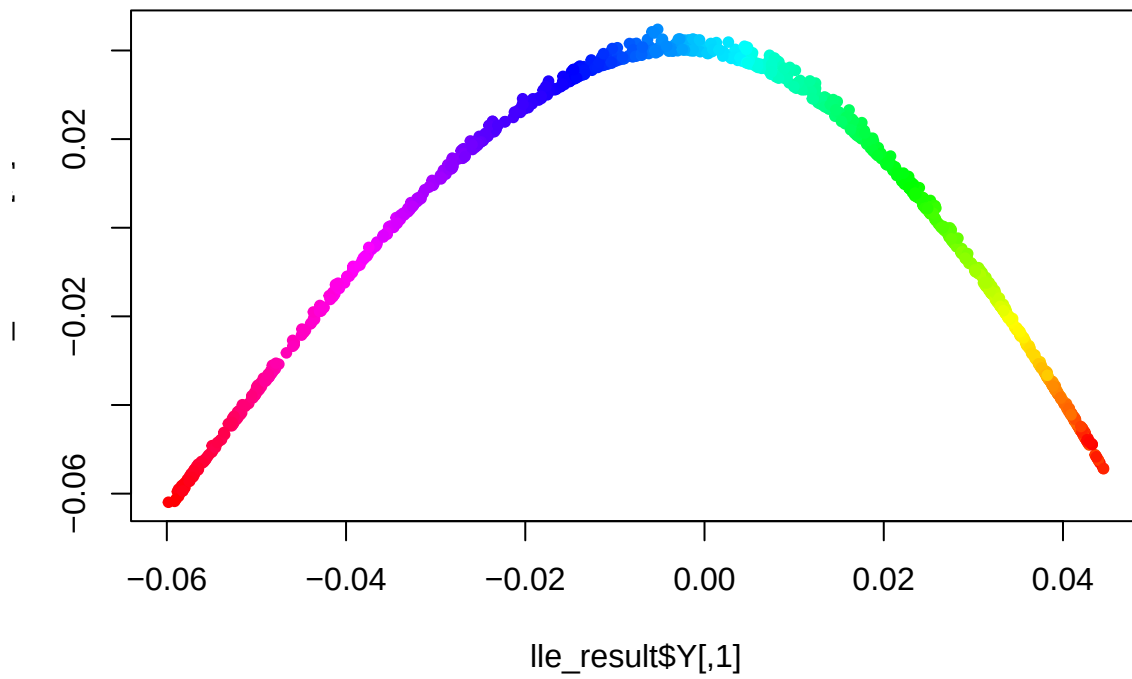


```
# === Isomap ===
isomap_input <- dimRedData(swiss[, 1:3])
isomap_result <- embed(isomap_input, "Isomap", knn = 10)
plot(isomap_result@data@data[, 1:2],
     col = colors, pch = 20, main = "Isomap")
```

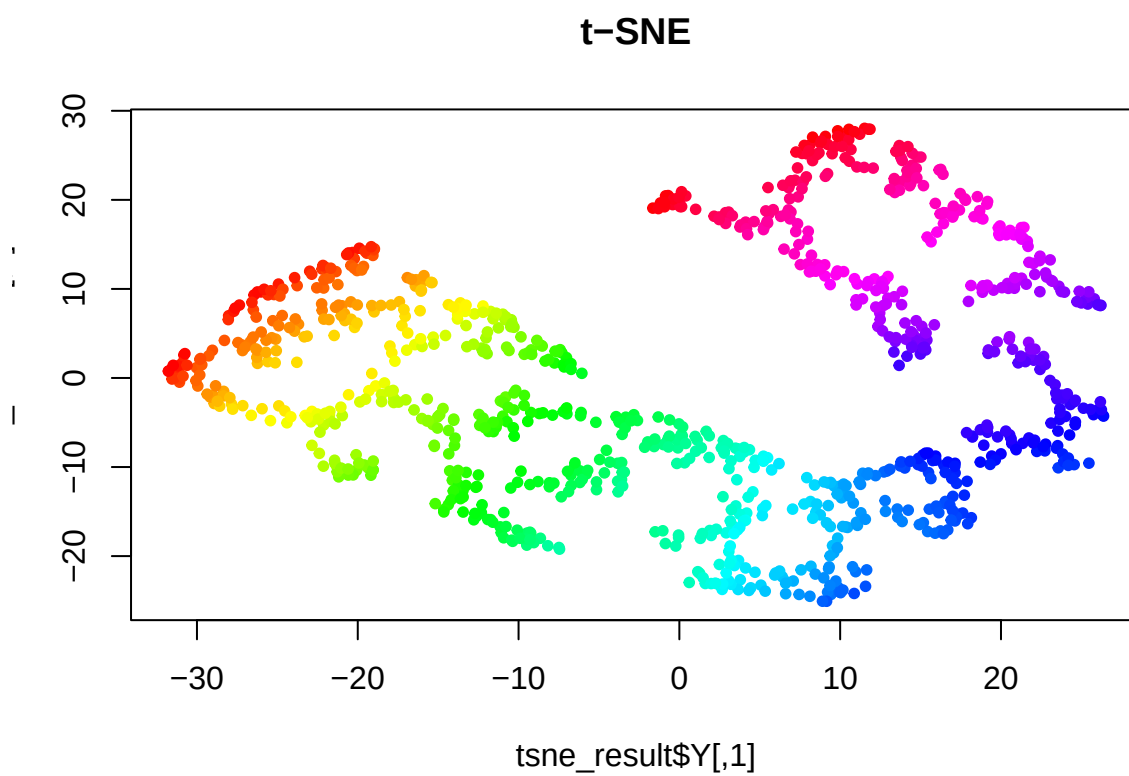


```
# === LLE ===
lle_result <- do.lle(as.matrix(swiss[, 1:3]),
                    ndim = 2,
                    type = c("knn", 12),
                    preprocess = "center")
plot(lle_result$Y,
     col = colors, pch = 20, main = "LLE")
```

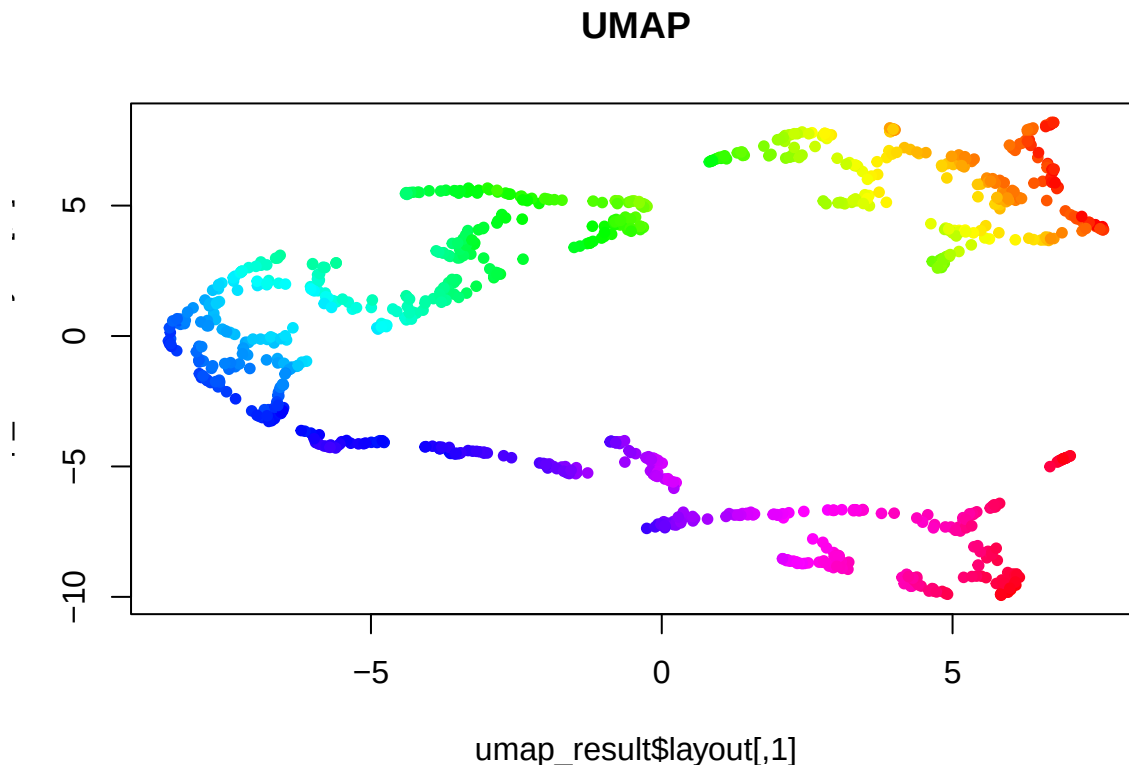
LLE



```
# === t-SNE ===  
tsne_result <- Rtsne(swiss[, 1:3], dims = 2, perplexity = 30)  
plot(tsne_result$Y,  
     col = colors, pch = 20, main = "t-SNE")
```



```
# === UMAP ===  
umap_result <- umap(swiss[, 1:3])  
plot(umap_result$layout,  
     col = colors, pch = 20, main = "UMAP")
```

Discuss how these methods reflects the underlying “unrolled” structure of the Swiss roll.

Results and Discussion:

The Swiss roll is basically a long, flat sheet that’s been curled up in 3D space — so when we try to reduce its dimensions, the goal is to recover that original flat structure.

Out of all the methods, Isomap did the best job. It “unwrapped” the spiral into a smooth, rectangular 2D shape while keeping the color gradient in order. That tells us it preserved the real distances between points along the surface of the roll — even when those points looked far apart in 3D.

LLE also captured some of the structure, though the result looked more like a curved arc. It seems to preserve local neighborhoods well, but not the overall global layout.

t-SNE and UMAP both found patterns too — you can still see the color gradients — but they focus more on clustering than keeping the global shape. They’re good at showing local similarities but not necessarily at “unrolling” the whole thing like Isomap.

So overall, these methods helped us visualize the roll’s hidden 2D shape — especially Isomap — by reducing dimensions while keeping the essential structure of the data.